

SKILL WEEK6

Task 1: Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX_ARGS 50
#define MAX_LEN 500

void parse_command(const char *input)
{
    char args[MAX_ARGS][MAX_LEN];
    int argc = 0;
    int pos = 0;
    int i = 0;

    int in_single_quote = 0;
    int in_double_quote = 0;
    int escape = 0;

    while (input[i] != '\0')
    {
        char c = input[i];

        /* Handle escape character */
        if (escape)
        {
            if (pos < MAX_LEN - 1)
                args[argc][pos++] = c;

            escape = 0;
            i++;
            continue;
        }

        /* Handle backslash */
        if (c == '\\')
        {
            escape = 1;
            i++;
            continue;
        }

        if (c == ' ')
        {
            if (in_single_quote || in_double_quote)
            {
                if (pos < MAX_LEN - 1)
                    args[argc][pos++] = c;
            }
            else
            {
                if (argc < MAX_ARGS - 1)
                {
                    args[argc] = {0};
                    argc++;
                    pos = 0;
                }
            }
        }
        else
        {
            if (pos < MAX_LEN - 1)
                args[argc][pos++] = c;
        }
        i++;
    }

    if (pos > 0)
        args[argc][pos] = '\0';
    else
        args[0] = {0};
}
```

```

}

/* Handle single quotes */
if (c == '\'' && !in_double_quote)
{
    in_single_quote = !in_single_quote;
    i++;
    continue;
}

/* Handle double quotes */
if (c == '"' && !in_single_quote)
{
    in_double_quote = !in_double_quote;
    i++;
    continue;
}

/* Handle spaces */
if (isspace((unsigned char)c) &&
    !in_single_quote &&
    !in_double_quote)
{
    if (pos > 0)
    {
        args[argc][pos] = '\0';
        argc++;
        pos = 0;
    }

    i++;
    continue;
}

/* Store normal character */
if (pos < MAX_LEN - 1)

```

```

/* Handle trailing escape */
if (escape)
{
    args[argc][pos++] = '\\';
}

/* Check unmatched quotes */
if (in_single_quote || in_double_quote)
{
    printf("\nParser Error: Unmatched quote detected.\n");
    return;
}

/* Add final argument */
if (pos > 0)
{
    args[argc][pos] = '\0';
    argc++;
}

printf("\n--- Parser Output ---\n");

if (argc == 0)
{
    printf("No arguments found.\n");
    return;
}

printf("Number of arguments: %d\n", argc);

for (int j = 0; j < argc; j++)
{
    printf("Argument %d: [%s]\n", j + 1, args[j]);
}

```

```

int main()
{
    char input[MAX_LEN];

    printf("=====\n");
    printf(" Process Escape Sequence Parser\n");
    printf("=====\n");

    printf("\nEnter a command:\n> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        printf("Input error.\n");
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    parse_command(input);

    return 0;
}

```

```
laasya@laasya:~/skillweek6$ nano task1.c
laasya@laasya:~/skillweek6$ gcc task1.c -o task1
laasya@laasya:~/skillweek6$ ./task1
=====
Process Escape Sequence Parser
=====

Enter a command:
> hello world test

--- Parser Output ---
Number of arguments: 3
Argument 1: [hello]
Argument 2: [world]
Argument 3: [test]
-----
laasya@laasya:~/skillweek6$ |
```

Task 2: Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_ARGS 20
#define MAX_LEN 100

int main()
{
    char command[MAX_LEN];
    char *args[MAX_ARGS];
    char *token;
    int argc = 0;

    printf("=====\n");
    printf(" Child Process Command Launcher\n");
    printf("=====\n");

    printf("\nEnter a command with arguments:\n> ");

    if (fgets(command, sizeof(command), stdin) == NULL)
    {
        printf("Input error.\n");
        return 1;
    }

    command[strcspn(command, "\n")] = '\0';

    /* Split command into arguments */
    token = strtok(command, " ");

    while (token != NULL && argc < MAX_ARGS - 1)
    {
        args[argc] = token;

```

```
if (argc == 0)
{
    printf("No command entered.\n");
    return 1;
}

printf("\nParent Process PID: %d\n", getpid());

pid_t pid = fork();

if (pid < 0)
{
    perror("fork failed");
    return 1;
}

if (pid == 0)
{
    /* Child process */
    printf("Child Process PID: %d\n", getpid());
    printf("Executing command: %s\n\n", args[0]);

    execvp(args[0], args);

    /* This executes only if execvp fails */
    perror("Execution failed");
    exit(1);
}
else
{
    /* Parent process */
    int status;

    printf("Parent waiting for child process...\n");

    if (waitpid(pid, &status, 0) == -1)
```

```

}
else
{
    /* Parent process */
    int status;

    printf("Parent waiting for child process...\n");

    if (waitpid(pid, &status, 0) == -1)
    {
        perror("waitpid failed");
        return 1;
    }

    if (WIFEXITED(status))
    {
        printf("\nChild process exited with status: %d\n",
            WEXITSTATUS(status));
    }
    else
    {
        printf("\nChild process did not exit normally.\n");
    }
}

printf("Parent process finished.\n");

return 0;
}

```

```
laasya@laasya:~/skillweek6$ nano task2.c
laasya@laasya:~/skillweek6$ gcc task2.c -o task2
laasya@laasya:~/skillweek6$ ./task2
=====
Child Process Command Launcher
=====

Enter a command with arguments:
> ls

Parent Process PID: 2403
Parent waiting for child process...
Child Process PID: 2406
Executing command: ls

task1 task1.c task2 task2.c

Child process exited with status: 0
Parent process finished.
laasya@laasya:~/skillweek6$ ./task2
=====
Child Process Command Launcher
=====

Enter a command with arguments:
> echo Hello Ubuntu

Parent Process PID: 2410
Parent waiting for child process...
Child Process PID: 2411
Executing command: echo

Hello Ubuntu

Child process exited with status: 0
Parent process finished.
laasya@laasya:~/skillweek6$ |
```